Aula 3

Criação de banco de dados, tabelas e usuários

Professor Me. Thiago de Jesus Inocêncio
thiago.inocencio@fatec.sp.gov.br

Criando usuários

Criação de usuários

Nesta aula iremos aprender como criar usuários para acessar nossos bancos de dados.

Para criar o primeiro usuário, é preciso entrar no banco de dados com o usuário root:

mysql -u root –p

Em seguida, digitar o seguinte comando que irá criar um usuário identificado como ‘usuario’ e com a senha é ‘senha’:

CREATE USER ‘usuario' **IDENTIFIED BY** ‘senha’;

O resultado deve ser igual à este: ***Query OK, 0 rows affected (0.014 sec)***

Tal resultado indica que o usuário foi criado com sucesso.

Testando o usuário criado

Para testar se realmente o usuário foi criado, é preciso sair do mariadb digitando:

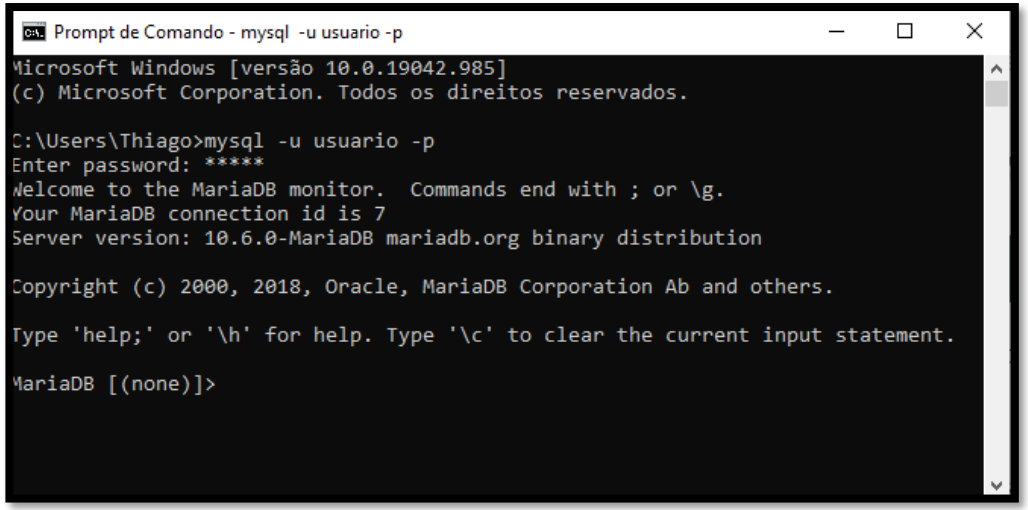
exit;

Em seguinda, deve-se logar usando as credenciais do novo usuário:

mysql -u usuario -p

Enter pasword*: [DIGITE A SENHA CRIADA]*

Algo semelhante a seguinte mensagem deve aparecer:



Criando usuários a partir de um IP

O usuário criado anteriormente poderá acessar de qualquer máquina com qualquer IP, isto é, qualquer máquina que tenha possibilidade de se conectar por rede com a máquina que está hospedando o mariadb.

Em alguns casos precisamos limitar o acesso de um usuário, por exemplo, permitir que um determinado usuário só possa acessar o banco a partir de uma máquina com IP específico.

Para isso, ao criar um usuário podemos selecionar de qual IP esse usuário pode entrar no banco de dados:

CREATE USER ‘usuario2’@’ip_usuario’ **IDENTIFIED BY** ‘senha2’;

CREATE USER ‘usuario2’@192.168.0.17 **IDENTIFIED BY** ‘senha2’;

Testando o usuário criado com IP

Para testar se realmente se o usuário2 foi criado, é preciso sair do mariadb digitando. Nesse caso vamos tentar fazer acesso ao servidor de uma máquina com IP 192.168.0.17. No servidor de IP 192.168.0.25

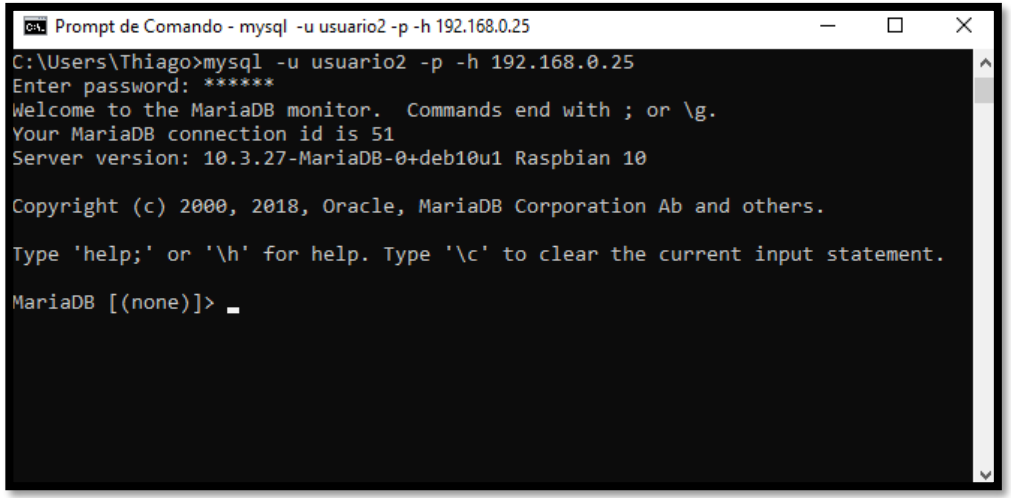
exit;

Em seguinda, deve-se logar usando as credenciais do novo usuário:

mysql -u usuario2 -p –h 192.168.0.25

Enter pasword*: [DIGITE A SENHA CRIADA]*

Algo semelhante a seguinte mensagem deve aparecer:



Gerenciando permissões

Permissões de banco de dados

Ao logar em um dos usuários e digitar o comando **SHOW DATABASES**, observa-se que apenas um banco de dados é listado.

Isso porque criamos o usuário, mas não atribuímos nenhuma permissão aos demais banco de dados.

Para dar permissão geral aos novos usuários ao banco de dados employees que criamos na aula anterior, digitamos o seguinte comando:

GRANT ALL ON employees.* to ‘usuario2’@192.168.0.17 **IDENTIFIED BY** ‘senha2' **WITH GRANT OPTION**;

Podemos concede permissão para o usuario2 acessar todos os bancos de dados com o seguinte commando:

GRANT ALL ON *.* to ‘usuario2’@192.168.0.17 **IDENTIFIED BY** ‘senha2' **WITH GRANT OPTION**;

Permissões de banco de dados

No slide anterior, concedemos permissões totais ao usuário 2, entretanto nem sempre isso é desejado. Em muitos casos, queremos apenas conceder permissões específicas para cada usuário. Temos as seguintes opções:

ALL – Concede acesso total a um banco de dados;
CREATE - Permitir que o usuário crie banco de dados e tabelas;
DELETE – Permitir que o usuário exclua linha de um banco de dados;
DROP – Permitir que o usuário 'DROPE' banco de dados e tabelas;
EXECUTE – Permitir que o usuário execução rotinas como funções e store procedures;
GRANT OPTION – Permitir que o usuário dê ou remova permissões de outros usuários;
INSERT – Permitir que o usuário faça inserções em tabelas;
SELECT – Permitir que o usuário faça consultas em um tabela;
SHOW DATABASES- Permitir que o usuário veja a lista de banco de dados existentes no SGBD;
UPDATE – Permitir que o usuário atualize linhas em banco de dados;

Permissões de banco de dados - exemplo

Vamos por exemplo dar permissão apenas de realizar consultas para o usuário2 na tabela salaries do banco de dados employees;

GRANT SELECT ON employees.salaries to ‘usuario2’@192.168.0.17 **WITH GRANT OPTION**;

Assim, o usuario2 ao fazer login da máquianA de IP 192.168.0.17, poderá consultar as informações presents na tabela salaries do banco de dados employees;

É uma boa prática reiniciar todas as permissões após realizar a alteração de algum usuário:

FLUSH PRIVILEGES;

Removendo usuários e permissões

Para revogar as permissões dados a um usuário, utilizamos o mesmo comando alterando apenas o termo GRANT para REVOKE, e TO para FROM por exemplo:

REVOKE SELECT ON employees.salaries **FROM** ‘usuario2’@192.168.0.17;

Para remover o usuário é bem simples. Podemos usar o comando **DROP USER** <nome_do_usuario>

DROP USER ‘usuario’;
DROP USER ‘usuario2’@192.168.0.17;

Gerenciando roles

Gerenciando roles

As **roles** no MariaDB é uma coleção nomeada de privilégios. Assim como contas de usuário, **roles** podem ter privilégios concedidos a elas e revogados delas.

Para criar novas **roles**, use a instrução **CREATE ROLE**:

CREATE ROLE ‘bd_developer', ‘bd_read', ‘bd_write’;

Para atribuir privilégios às funções, podemos executar instruções **GRANT** usando a mesma sintaxe usada para atribuir privilégios a contas de usuário:

GRANT ALL ON employees.* **TO** ‘bd_developer’;
GRANT SELECT ON employees.* **TO** ‘db_read’;
GRANT INSERT, UPDATE, DELETE ON employees.* **TO** ‘db_write’;

FLUSH PRIVILEGES;

Gerenciando roles

Agora suponha que inicialmente você precise de uma conta de desenvolvedor, duas contas de usuário que precisam de acesso somente leitura e uma conta de usuário que precisa de acesso de leitura/escrita. Use **CREATE USER** para criar as contas:

CREATE USER ‘dev1’@’%’ **IDENTIFIED BY** ‘123’;
CREATE USER ‘read_user1’@’%’ **IDENTIFIED BY** ‘123’;
CREATE USER ‘read_user2’@’%’ **IDENTIFIED BY** ‘123’;
CREATE USER ‘rw_user1’@’%’ **IDENTIFIED BY** ‘123’;

FLUSH PRIVILEGES;

Gerenciando roles

Para atribuir a cada conta de usuário os privilégios necessários, você poderia usar instruções GRANT da mesma forma que acabou de ser mostrado, mas isso requer a enumeração de privilégios individuais para cada usuário.

```
GRANT 'app_developer' TO 'dev1'@’%’;
GRANT 'app_read' TO 'read_user1'@’%', 'read_user2'@’%’;
GRANT 'app_read', 'app_write' TO 'rw_user1'@’%’;

FLUSH PRIVILEGES;
```

Para ver as permissões concedidas para o usuário dev1, utilize o commando:

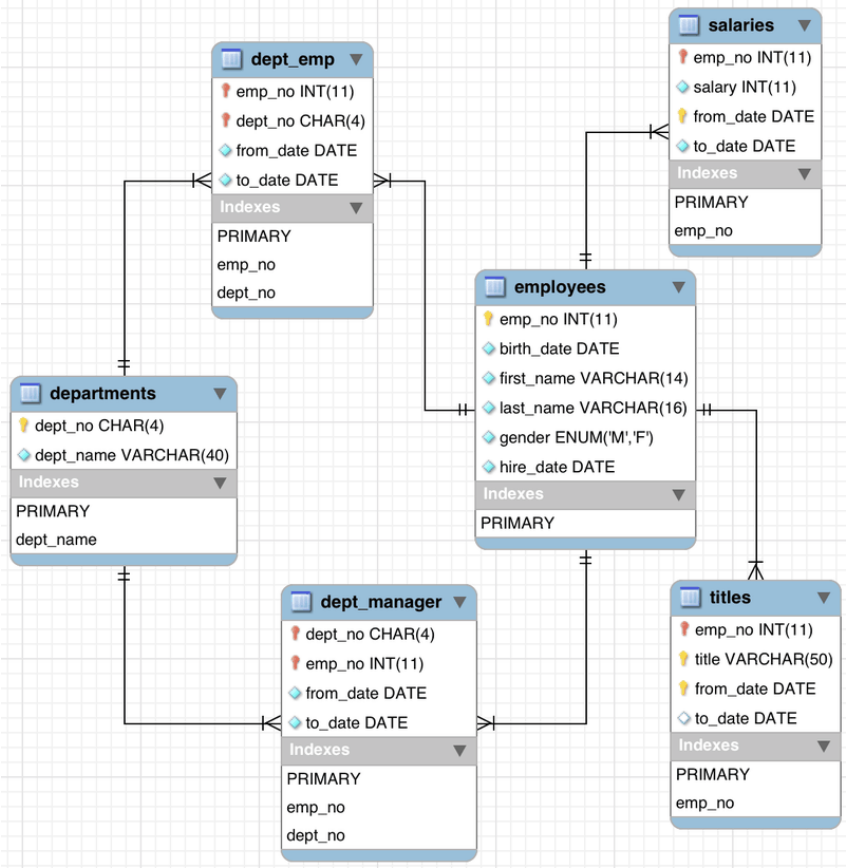
```
SHOW GRANTS FOR 'dev1'@’%';
```

Criação de banco de dados

Banco de dados employees e suas tabelas

Tomando como exemplo a tabela importada na aula anterior, essas tabelas foram inseridas dentro do banco de dados chamado employees.

Nesta aula vamos criar todas essas tabelas passo a passo, inclusive o banco de dados.



Criação do banco de dados employees

Antes de iniciar a criação do banco de dados, vamos nos certificar que a importação feita na aula anterior seja removida, para isso execute no terminal do banco de dados:

DROP DATABASE employees;

Após a execução do comando, deve ser exibido o seguinte resultado:

Query OK, 8 rows affected (0.181 sec)

Nesse caso, como fizemos o drop de um banco de dados, 8 tabelas desse banco também foram excluídas.

Criação do banco de dados employees

Depois de excluído o banco de dados, executamos o seguinte comando para criar um novo:

CREATE DATABASE employees;

Em negrito está o comando SQL responsável pela criação do banco.

O resultado deste comando deve ser algo como:

Query OK, 1 row affected (0.001 sec)

Indicando que o banco de dados foi criado com sucesso;

Ver bando de dados criados

Podemos verificar se o banco de dados foi criado com sucesso de duas formas:

1. Executando o comando: SHOW DATABASES;
2. Executando o comando: USE employees;

No primeiro caso será exibida uma lista dos bancos de dados existentes (conforme as permissões do usuário). O banco de dados employees deve ser mostrado.

Na segunda opção, o resultado deve ser Database changed, indicando a mudança para o banco de dados employees.

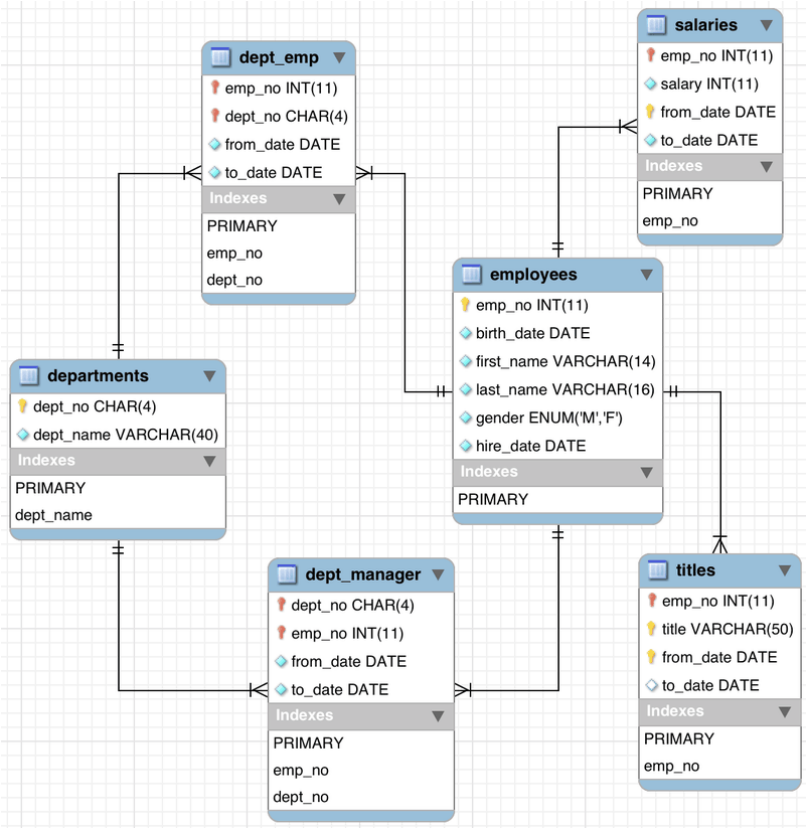
Criação de tabelas

Criação de banco de dados

Uma vez criado o banco de dados, podemos selecioná-lo com o comando **USE** e, em seguida, criar as tabelas.

Novamente, as tabelas do banco de dados employees podem ser vistas no DER ao lado.

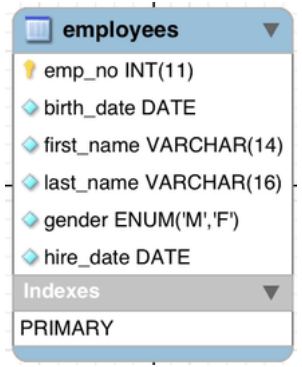
Com base nos tipos de dados vistos na aula anterior, podemos criar essas tabelas e suas relações por meio do comando **SQL CREATE TABLE**;

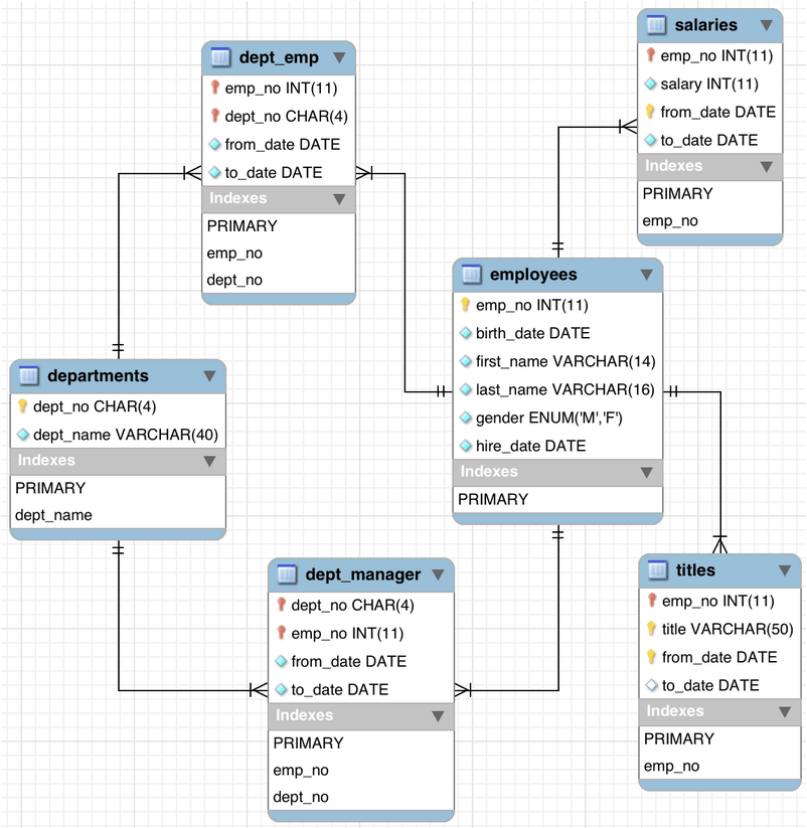


Criação de banco de dados

Ao observar o DER, percebemos que as tabelas employees e departments são as únicas que não contém chaves estrangeiras de outras tabelas.

Nesse sentido, iniciaremos pela criação dessas tabelas.



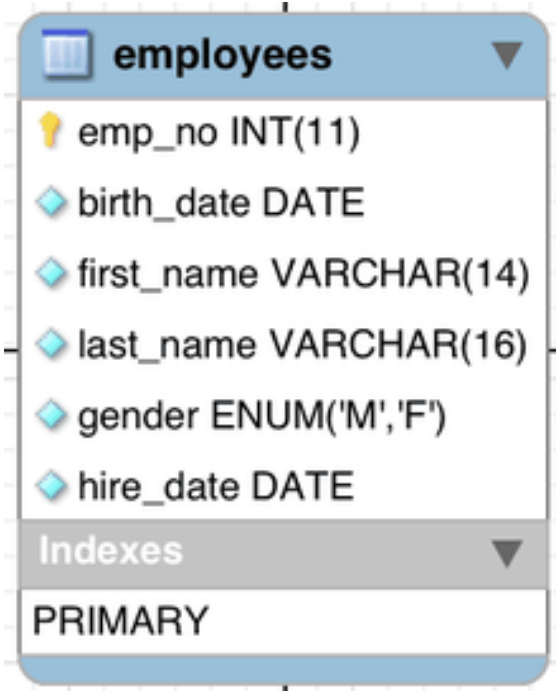


Criação da tabela employees

Antes de criar a tabela, precisamos selecionar o banco de dados que a tabela vai pertencer. Nesse caso, digitamos:

USE employees;

O próximo passo é utilizar o comando **CREATE TABLE** para criar a tabela employees.



Comando CRETE TABLE

A sintaxe do comando **CREATE TABLE** é a seguinte:

CREATE TABLE nome_da_tabela **(**<nome_campo_1> <tipo> <restrições_campo_1>, <nome_campo_2> <tipo> <restrições_campo_1>, ... , <nome_campo_3> <tipo> <restrições_campo_1>**)**

Exemplo: Vamos criar uma tabela de usuários simples com três campos id_usuario, primeiro_nome e idade.

CREATE TABLE usuário (id_usuario **INT PRIMARY KEY**, primeiro_nome **VARCHAR**(30), idade **TINYINT**);

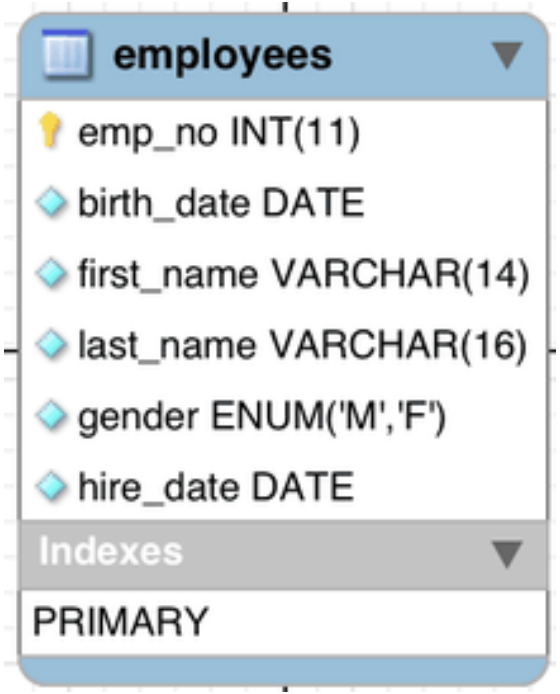
Criação da tabela employees

Agora que sabemos como criar uma tabela, vamos criar a consulta SQL para criar a tabela employees:

```
CREATE TABLE employees (
 emp_no INT PRIMARY KEY,
 birth_date DATE,
 first_name VARCHAR(14),
 last_name VARCHAR(16),
 gender ENUM(‘M’, ‘F’),
 hire_date DATE
)
```

Para verificar se a tabela foi criada, digite os comandos:

```
desc employees;
```



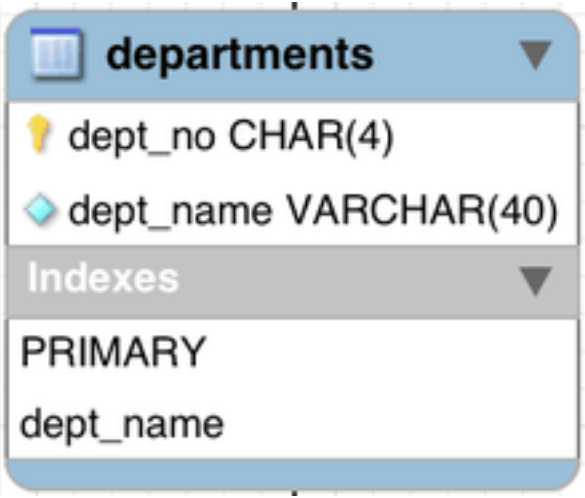
Criação da tabela departments

vamos criar a consulta SQL para criar a tabela departments:

```
CREATE TABLE departments (
 dept_no CHAR(4) PRIMARY KEY,
 dept_name VARCHAR(40),
 INDEX (dept_name)
);
```

Para verificar se a tabela foi criada, digite os comandos:

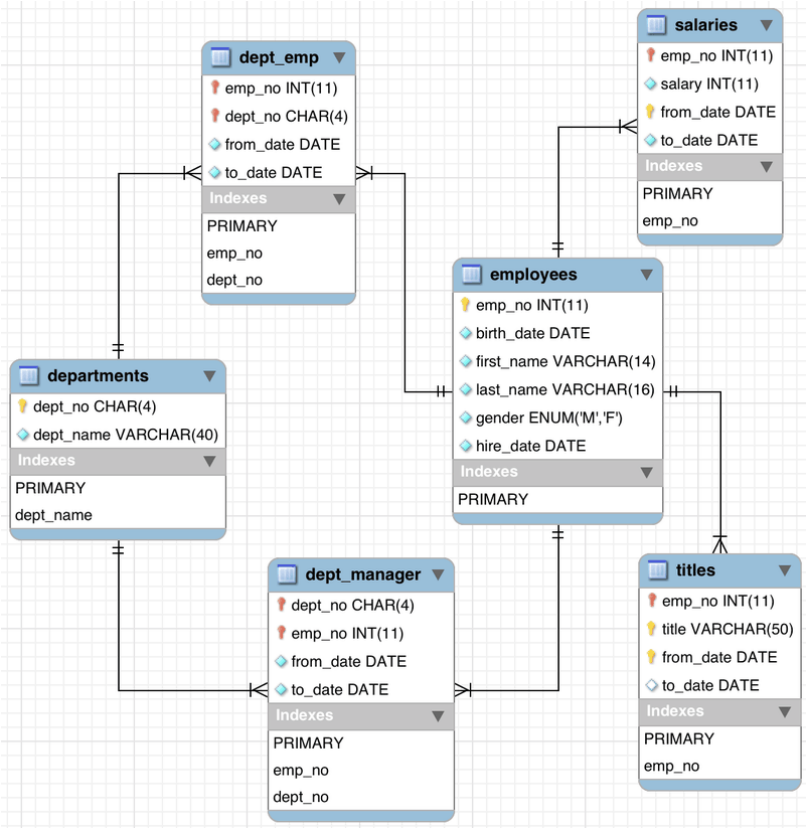
```
desc departments;
```



Criação das tabelas salaries e titles

Vamos criar agora as tabelas salaries e titles que estão ligadas a tabela employees.

Perceba que tanto a tabela salaries quanto titles possuem a chave estrangeira emp_no que as relaciona com a tabela employees.



Criação da tabela salaries

vamos criar a consulta SQL para criar a tabela salaries:

```
CREATE TABLE salaries (
 emp_no INT(11),
 salary INT(11),
 from_date DATE,
 to_date DATE,
 FOREIGN KEY (emp_no) REFERENCES employees(emp_no)
);
```

Para verificar se a tabela foi criada, digite os comandos:

```
desc salaries;
```

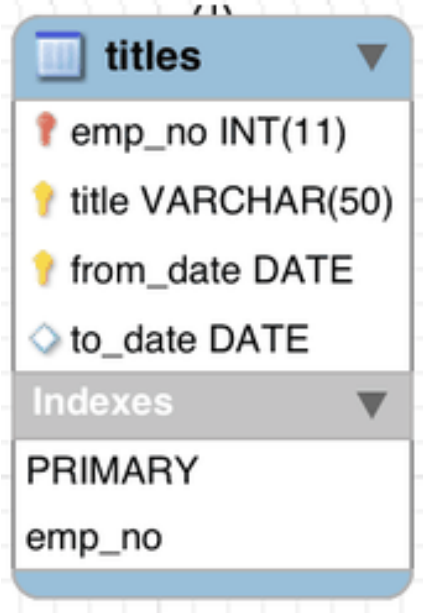
Criação da tabela titles

vamos criar a consulta SQL para criar a tabela salaries:

```
CREATE TABLE titles (
 emp_no INT(11),
 title VARCHAR(50),
 from_date DATE,
 to_date DATE,
 FOREIGN KEY (emp_no) REFERENCES employees(emp_no)
);
```

Para verificar se a tabela foi criada, digite os comandos:

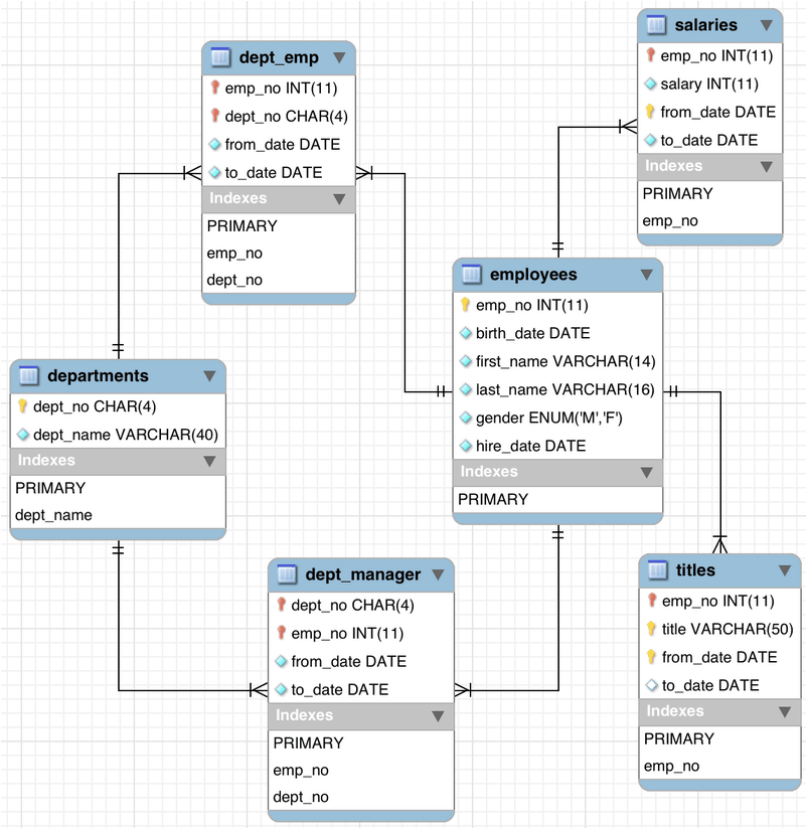
```
desc salaries;
```



Criação das tabelas dept_emp e dept_manager

Vamos criar agora as tabelas dept_emp e dept_manager que estão ligadas às tabelas employees e departments.

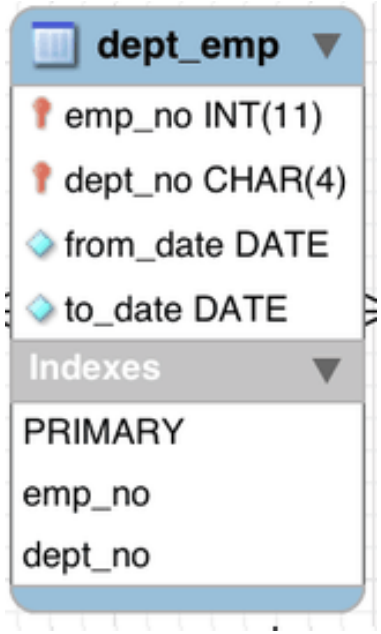
Perceba que tanto a tabela dept_emp quanto dept_manager possuem as chaves estrangeiras emp_no e dept_no que as relacionam com as tabelas employees e departments ao mesmo tempo.



Criação da tabela dept_emp

vamos criar a consulta SQL para criar a tabela dept_emp:

```
CREATE TABLE dept_emp (
 emp_no INT(11),
 dept_no CHAR(4),
 from_date DATE,
 to_date DATE,
 FOREIGN KEY (emp_no) REFERENCES employees(emp_no),
 FOREIGN KEY (dept_no) REFERENCES departments(dept_no)
);
```



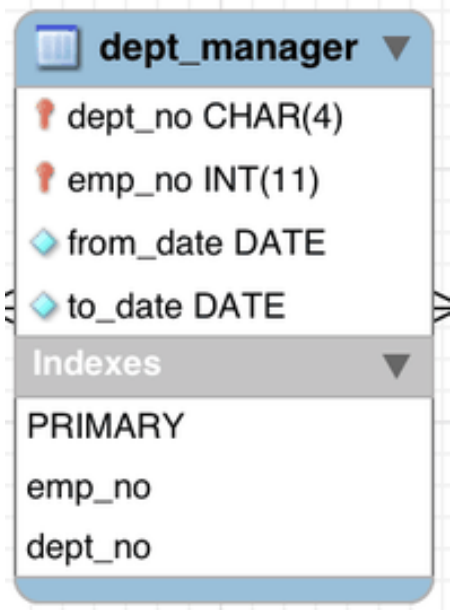
Para verificar se a tabela foi criada, digite os comandos:

```
desc dept_emp;
```

Criação da tabela dept_emp

vamos criar a consulta SQL para criar a tabela dept_manager:

```
CREATE TABLE dept_manager (
 emp_no INT(11),
 dept_no CHAR(4),
 from_date DATE,
 to_date DATE,
 FOREIGN KEY (emp_no) REFERENCES employees(emp_no),
 FOREIGN KEY (dept_no) REFERENCES departments(dept_no)
);
```



Para verificar se a tabela foi criada, digite os comandos:

```
desc dept_manager;
```

Exercício

Criação de banco de dados

Com base no que foi exposto em aula, faça:

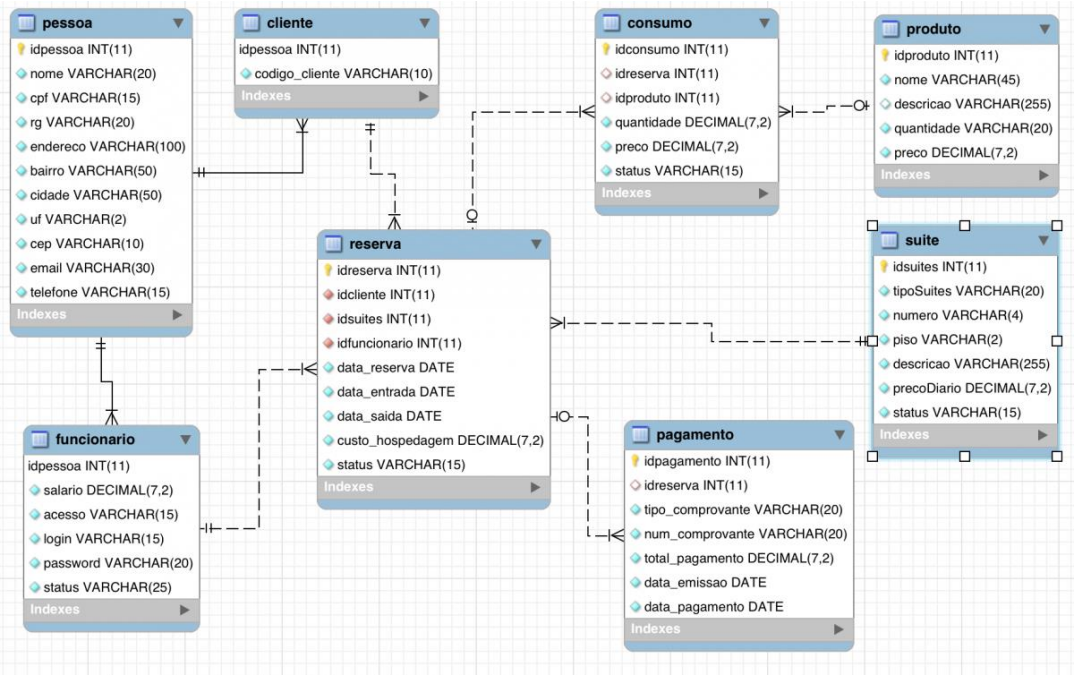
a) Crie o banco de dados chamado **fatel**;

b) Crie a tabelas conforme o diagrama com suas respectivas relações;

c) Crie uma **ROLE** chamada **fatel_read** que conceda acesso apenas de leitura para o banco **fatel**;

d) Crie um usuário chamado **fateluser** e atribua a ela a role **fatel_read**;

e) Entre no banco com o usuário **fateluser** e interaja com o banco **fatel**;



Referências

SILBERSCHATZ, HENRY, A.; KORTHS, H. F.; SUDARSHAN, S. Sistema de banco de dados. 5. ed. Rio de Janeiro. Campus, 2006.

ELMASRI, R.; NAVATHE, S. B. Sistemas de banco de dados: fundamentos e aplicações. 4.ed. São Paulo. Pearson, 2005.

W3schools. MySQL Data Types. https://www.w3schools.com/mysql/mysql_create_db.asp